Policy Impact Assistant

Handbook for staff, administrators and engineers

Find every policy affected by a change to another, with the evidence for each, on your own hardware. This handbook covers how it works, daily use, installation, operations, security, review and evaluation.

secretary-keeper version 0.1 (hermes-langgraph branch), October 2026. MIT License.

Contents

1 **Architecture** 3

How it works and why it's built this way

For: Everyone

2 **How to use it** 18

Ask questions and read the results; add policies, groups and models

For: Staff and administrators

3 **Setup** 22

Install it on the GPU server

For: Engineers

4 **Operations** 28

Run it day to day, upgrade it, fix problems

For: Administrators and engineers

5 **Security and safety practices** 32

Keep policy text contained and the system safe

For: Everyone

6 **Review queue and judge training** 37

Let people check answers, then teach the judge from them

For: Administrators, reviewers and engineers

7 **Evaluation** 43

Measure whether it finds the right policies

For: Policy owners and engineers

8 **Testing** 46

Run and understand the tests

For: Engineers

Chapter 1

Architecture

How it works and why it's built this way.

For: Everyone

This is the **hermes + langgraph** version. People chat with an assistant (Hermes Agent). The assistant decides which tool to call. The tools talk to policy-api, which runs each check as a **LangGraph graph** that saves its progress to Postgres after every step. When a check ends with "maybe" sections, it can pause until a person decides them in the chat. Other versions of the product are described in the README.

The job

When a head policy changes, find every policy in a chosen group that now conflicts with it, repeats its old wording, points to a section that moved, or uses a defined term whose meaning changed. Missing an affected policy is the expensive mistake, so the design favours **recall** (finding everything) over precision (flagging nothing extra), and it puts a person in the loop for anything uncertain.

The core rule

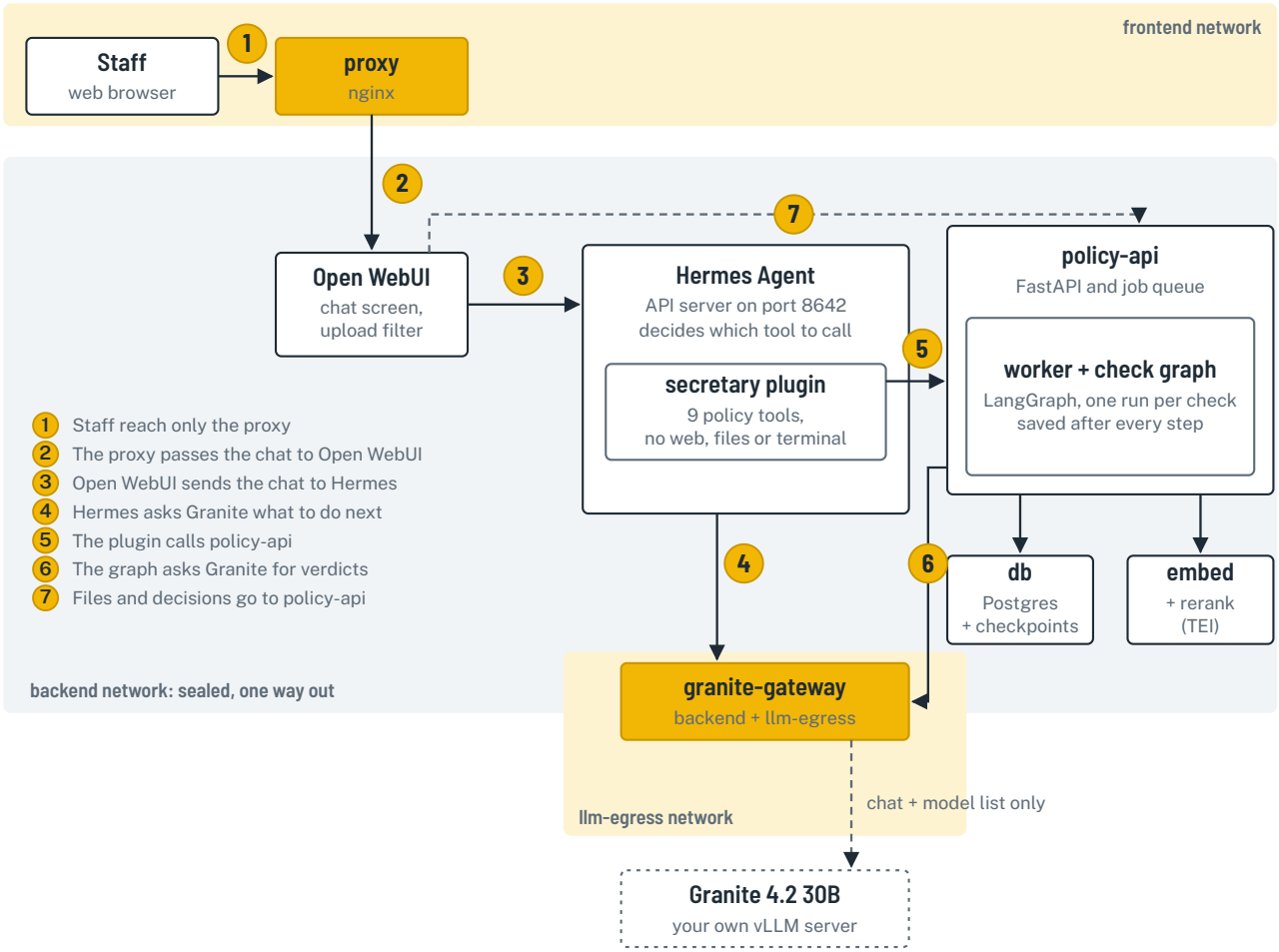
The chat model never decides impact itself. Hermes works out what the person is asking, calls a tool, and passes on the result. The graph inside policy-api makes every decision, and code builds every number, table and spreadsheet. The chat answer and the Excel file therefore always agree, and every decision can be traced in the audit trail.

Two more rules follow from it:

- **The graph's route is chosen by code, never by a model.** Models only answer one question about one pair of sections. They never pick the next step.
- **The model cannot record a review decision.** There is no tool for it. A decision exists only when a person types it, as their own message, and trusted code (the upload filter) reads it.

Hermes also has no other source of facts. It has no web search, no files, no terminal, no memory and no MCP servers. If the tools do not return it, the assistant says "Our policies don't cover that".

The parts in one picture



The request path. People reach only the proxy; Hermes and the check graph reach Granite only through the gateway.

Read the numbers 1 to 7 in the legend. Solid lines are requests; dashed lines are attached files, typed review decisions and the single way out to the Granite server.

Request flow, step by step

A. A question about what a policy says

Example: "What do we require for lockout devices?"

1. The person types in Open WebUI, reached through the `proxy`.
2. Open WebUI sends the chat to Hermes as an OpenAI-style request for the model `policy-assistant`, signed with `HERMES_API_KEY`.
3. Hermes builds a prompt from its rules (`hermes/SOUL.md`), the list of tools it may use, and the chat so far. It sends the prompt to Granite through `granite-gateway`.
4. Granite replies with a tool call, here `search_policies(question=...)`.

5. The plugin calls `GET /search` on policy-api. policy-api embeds the question (TEI `embed`), finds the nearest sections in Postgres (pgvector), and lets the reranker put them in order. The best 8 (up to 20) come back, each cut to 1,500 characters.
6. The plugin wraps each section in `<<<` and `>>>` so the model treats it as wording, never as instructions, and hands the text to Hermes. Hermes sends it to Granite.
7. Granite writes the answer and cites each fact as `[policy §section]`. Hermes returns it to Open WebUI.

If nothing matches, the tool says so and the assistant answers that the policies do not cover it. To read a whole policy or one section, Granite calls `read_policy` (`GET /policies/{id}/text`).

B. A question with an attached document

1. The person attaches a file. Before the message leaves Open WebUI, the **upload filter** (`openwebui/upload_filter.py`, a global function) sends the file to `POST /uploads` on policy-api with `POLICY_API_KEY`.
2. policy-api reads the file in its own limited process, splits it into numbered sections, stores it, and guesses which stored policy it is a new version of.
3. The filter adds a note to the end of the person's message: `[Attached file ...: upload id ...]`. It also stops Open WebUI from cutting the file into its own search index.
4. Hermes sees the note. Its rules tell it to pass the upload id to the tools.

C. A policy change check, end to end

Example: "Which Safety policies are affected by the change to POL-007?" with the new version attached.

1. Granite calls `check_policy_change(policy_group, head_policy, upload_id)`.
2. The plugin finds the group (`GET /subsets`) and the head policy (`GET /policies`). If a name is not a clear match, the tool returns a question for the person instead of guessing.
3. The plugin sends `POST /runs`, recording the request as made by `assistant`. policy-api stores a `runs` row and a row in the `jobs` table, and answers **202 Accepted** with a run id.
4. A worker inside policy-api claims the job (`FOR UPDATE SKIP LOCKED`) with a 60-second lease and renews the lease while it works. It starts the **graph** for that run (next section). The graph's state is saved to Postgres after each node. Many Granite calls run at once (`JUDGE_CONCURRENCY`, default 16).
5. While the job runs, the plugin asks `GET /runs/{id}` every 3 seconds. The Hermes turn waits, so the person's chat shows a running answer.
6. When the run is `done`, policy-api returns the chat answer (built in code) and a signed Excel link that works for 72 hours. The plugin passes both to Hermes with an instruction to show them exactly, and a note for Hermes alone with the 8-character check number.
7. Granite relays the answer. If the person then asks for a PDF, Word file or charts, Granite calls `make_check_files`, which uses `POST /runs/{id}/files`.

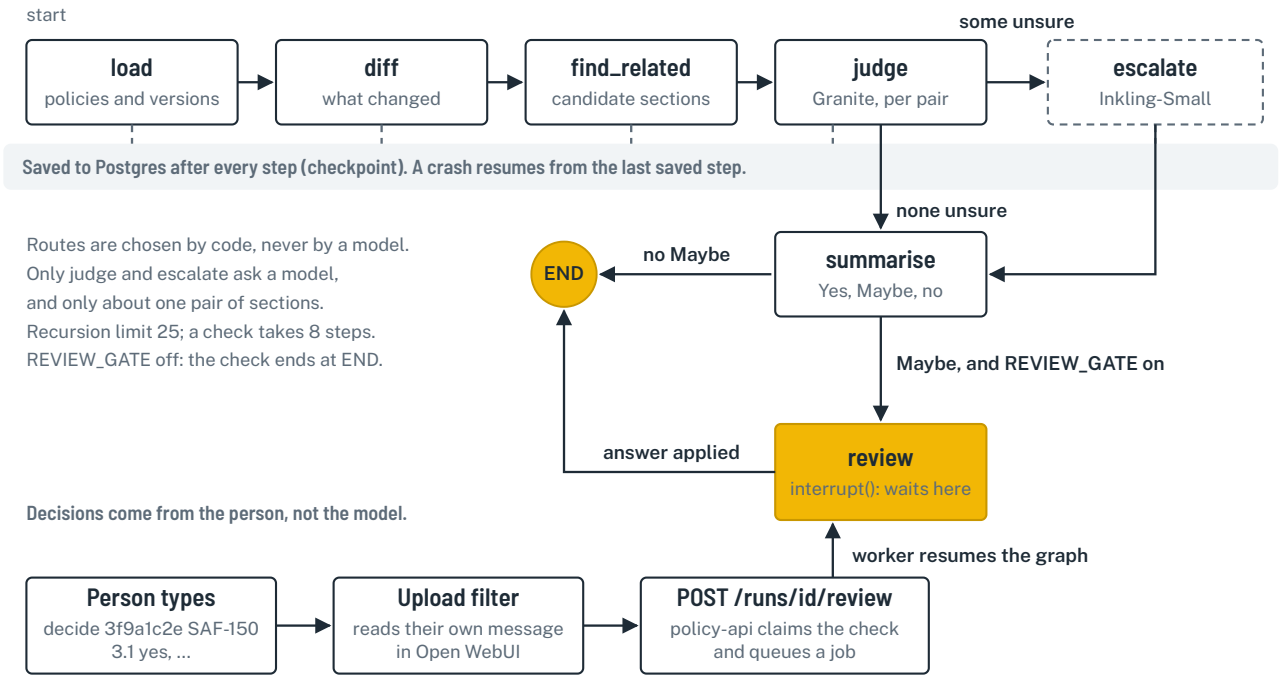
D. A check that pauses for a person

This happens only when `REVIEW_GATE=1` and the answer has at least one "maybe" section.

1. After `summarise`, the graph reaches the `review` node and calls LangGraph's `interrupt()`. The worker records the run as `needs_review`, with the list of "maybe" sections, and the job ends. The check now holds no worker and no lease; only its saved state waits in Postgres.
2. The plugin's next poll sees `needs_review`. It returns guidance to Hermes instead of an answer: show the list, show the check number (the first 8 characters of the run id), and tell the person to type a message that starts with `decide`, in the format below. The `<yes|no>` markers are placeholders the person replaces with their own answer.
3. The person types their whole message, for example `decide 3f9a1c2e SAF-150 3.1 yes, SAF-151 2.4 no`. (`decide 3f9a1c2e all no` answers every listed section.)
4. In Open WebUI, **before** the message goes to Hermes, the upload filter reads it. It accepts it only if the person's own latest message starts with the word `decide`. The filter takes the person's name from Open WebUI and calls `POST /runs/{check}/review` with `POLICY_API_KEY`.
5. policy-api checks that every listed section is really waiting, then **claims** the run with an atomic `needs_review → queued` update, so two answers sent together cannot both apply. It queues a `review_run` job and answers 202. A short check number is accepted here only among paused checks.
6. The filter adds a note to the person's message ("decisions were recorded", or why they were not). Hermes tells the person in plain words and calls `get_check_result`.
7. A worker runs the `review_run` job. It resumes the graph at the same thread with the person's answer. `review` turns each decided "maybe" into "Yes" (with "Confirmed by ") or drops it, recounts, and the graph ends. The run becomes `done` and the plugin returns the final answer.

Sections the person leaves out stay "Maybe". The check still ends, so it cannot be decided a second time. The decision changes the result of that check only. It is not written to the `judgements` rows used for training (review-and-training.md).

The check graph



One check as a graph. Code picks every route, a checkpoint is saved after each step, and a person's typed decision resumes a paused check.

Each check is one run of the graph in `policy_api/app/graph.py`. The graph is fixed. It has seven nodes, and code chooses every route.

Node	What it does	Uses a model?
`load`	Resolves the head policy, the two versions to compare (stored versions, or stored plus the upload) and the group's other policies. Stops with a plain reason if there is nothing to check	No
`diff`	Clause-level comparison of old and new head policy: modified, added, removed and renumbered sections, and defined terms touched. Saves each change	No
`find_related`	For each changed section, collects clauses in the group that cite the head policy, use a changed defined term, or are similar in meaning (embedding search, then the reranker keeps the top N)	TEI only
`judge`	Granite reads each (changed section, related clause) pair and returns yes / no / unsure, a reason, evidence quotes and a suggested edit, forced into a JSON schema	Granite
`escalate`	"Unsure" pairs go to Inkling-Small for a second opinion. Skipped if no escalation model is set up	Inkling-Small
`summarise`	Per policy: Yes if any section is "yes", Maybe if any is "unsure", otherwise not affected. Builds the result	No
`review`	Pauses (`interrupt`) until a person decides the "maybe" sections, then applies the answer	No

The `find_related` node keeps two kinds of clause that the reranker never cuts: those that cite the head policy and those that use a changed defined term. Explicit links are where most real impact is. The similarity search catches policies that overlap in meaning without citing the head policy.

Conditional edges. Three decisions, all made in code:

After	Goes to	When
`judge`	`escalate`	At least one pair is "unsure" and an escalation model is configured
`judge`	`summarise`	Otherwise
`summarise`	`review`	At least one row is "Maybe" and `REVIEW_GATE` is on
`summarise`	end	Otherwise

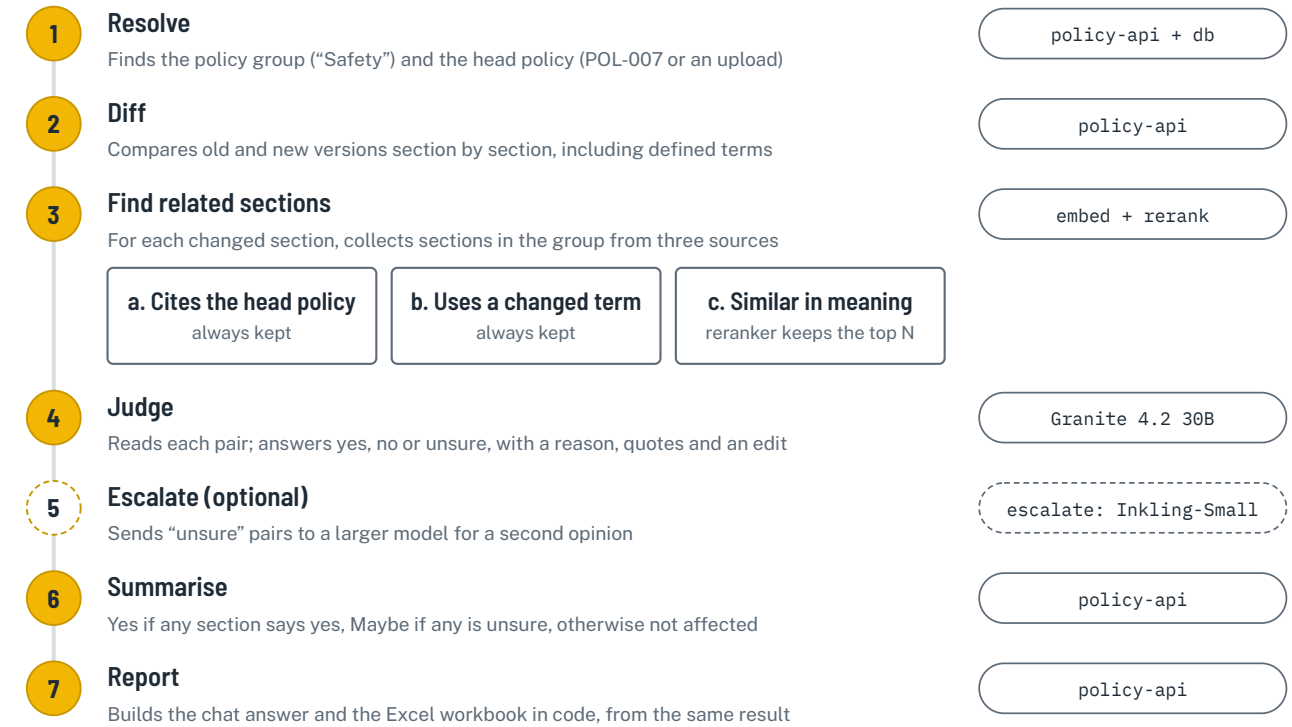
State (`RunState`). Plain JSON values only, so a saved checkpoint never depends on the code's classes.

Field	Holds
`run_id`	The run this graph belongs to (also the checkpoint thread id)
`head`, `labels`, `policy_ids`, `metas`	The head policy, the names of the two versions compared, the group's other policies and their details
`changes`	The changed sections, each with the id of its saved row
`pairs`	One entry per (change, related clause): the clause, how it was found, the verdict, and whether it was escalated
`errors`	How many pairs the model could not judge
`result`	The finished answer: rows, counts, not-affected list
`reviewed`	The person's answer, once the check has been decided

Checkpointer. `AsyncPostgresSaver` (langgraph-checkpoint-postgres 3.2.0, LangGraph 1.2.16) writes the state to the same Postgres database after every node (`durability="sync"`, so the write finishes before the next node starts). It creates its own tables on first start. The thread id is the run id. In demo mode and in tests it uses memory instead.

Recursion limit. 25 steps. A normal check takes at most 8, so reaching the limit means a bug, not a big check. The check then fails with "Something went wrong on our side" and the error is logged.

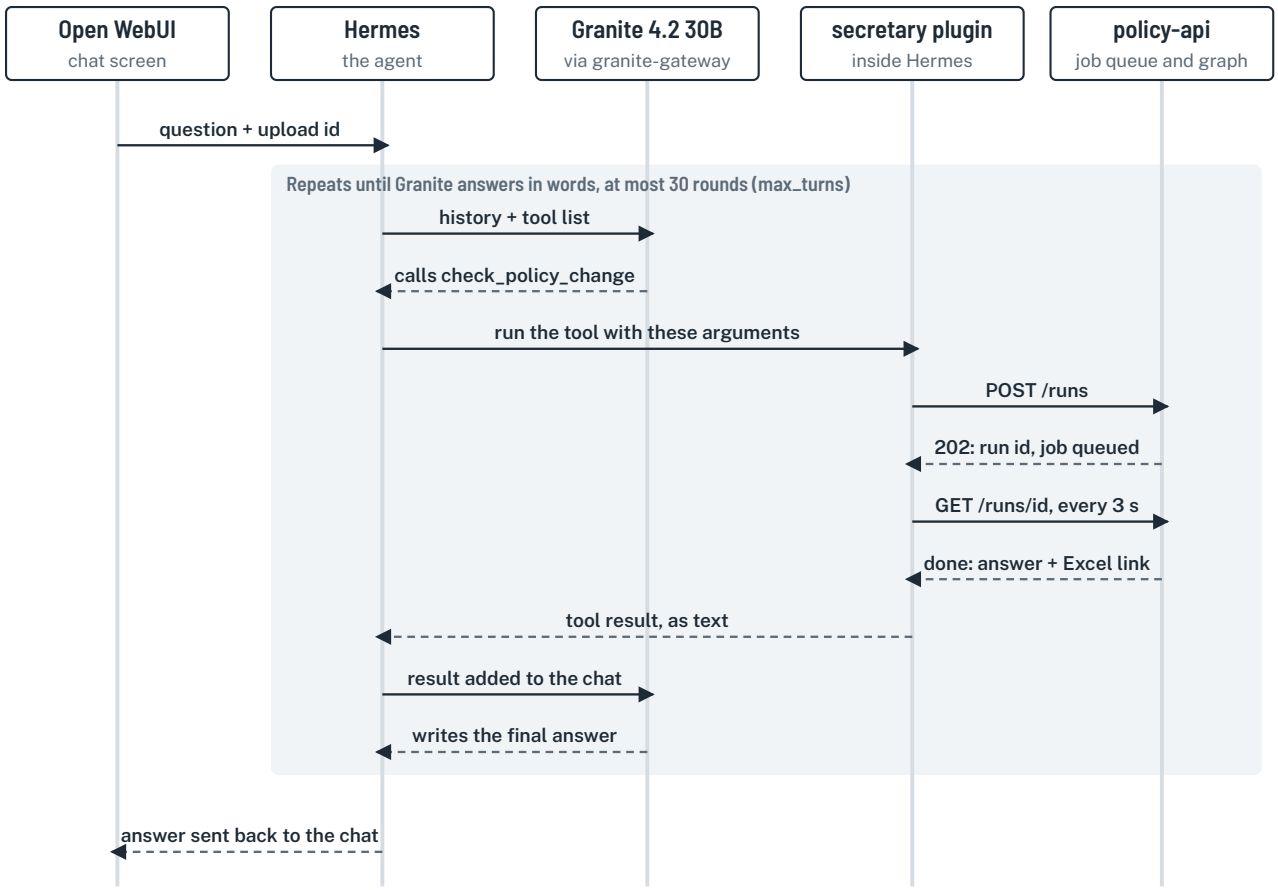
Resume. When a job is retried, the worker asks LangGraph for the run's saved state. If steps are left, the graph carries on from the last finished node. Only the node that was running is repeated, and its half-written verdicts are deleted first.



How one question is answered. Code makes every decision; the judge rules on one pair at a time.

The same steps in plain order. Steps 1 to 6 are the graph nodes in the table above (step 1 is `load`). Step 7 happens after the graph: the chat answer and Excel file are built from the saved result when someone asks for them.

The agent loop



One question, step by step. Granite proposes tool calls; the plugin runs them against policy-api; code makes every decision.

Hermes repeats the same small loop for every question: ask Granite, run the tool Granite picks, give Granite the result, and ask again. The loop ends when Granite answers in words instead of asking for a tool. Hermes stops it after `max_turns: 30` rounds.

The plugin returns plain text, not data. Each result starts with an instruction to Granite ("show this answer exactly as written", "answer only from these sections"). That is how numbers, policy IDs and links reach the person unchanged.

Components

Component	Image and version	Role	Networks
`proxy`	`nginx:1.27-alpine` (`NGINX_TAG`)	The only thing people reach: Open WebUI, plus report downloads and `/health`. Holds no data	backend + frontend
`open-webui`	`ghcr.io/open-webui/open-webui` (`OPENWEBUI_TAG`, pinned in `.env`)	Chat screen. Sends chats to Hermes. Runs the upload filter (files and review decisions). Trimmed: no web search, tools, memories or code running	backend

Component	Image and version	Role	Networks
`hermes`	`nousresearch/hermes-agent` (`HERMES_TAG`; tested with v0.22.3)	The assistant. OpenAI-compatible API on port 8642, model name `policy-assistant`. Runs the agent loop and the `secretary` plugin	backend
`policy-api`	Built from `policy_api/` (Python 3.12, FastAPI 0.115 or later; `APP_VERSION`). Uses LangGraph 1.2.16 and langgraph-checkpoint-postgres 3.2.0	The check graph, the job queue and worker, search, uploads, reports and downloads	backend
`db`	`pgvector/pgvector:pg17` (`PGVECTOR_TAG`)	Postgres with pgvector: policies, clauses, graph, runs, jobs, audit trail, and the graph's checkpoints	backend
`embed`, `rerank`	`text-embeddings-inference` (`TEI_TAG`) with Granite embedding and reranker models	Embeddings for similarity search; reranker to order and cut candidates	backend
`granite-gateway`	`nginx:1.27-alpine`	The backend's only way out. Passes chat and model-list calls to `GRANITE_URL`, adds the Granite key, checks TLS	backend + llm-egress
Granite 4.2 30B	Your own vLLM server, outside this stack (vLLM 0.20 or later)	The model behind Hermes' reasoning and the graph's verdicts	Reached only through the gateway
`escalate` (optional)	`vllm/vllm-openai` (`VLLM_TAG`), Inkling-Small	Second opinion on "unsure" pairs	backend
`extract` (optional)	`vllm/vllm-openai` (`VLLM_TAG`), gpt-oss-20b	Finds references the regex misses during ingest	backend
Review and training services (optional)	Argilla v2.8.0 and its helpers; `lora-trainer`; `judge-candidate`; `judge-tuned`	Review queue and fine-tuning (review-and-training.md)	backend

The LangGraph graph runs **inside** `policy-api`. It is not a separate container.

The Open WebUI tool file `openwebui/policy_impact_tool.py` is **not used** in this version. Nothing in `compose.yaml` or the setup steps installs it, and Open WebUI's tool servers are switched off. Hermes and its plugin do that job. The file stays in the repo for the `main` and `langgraph` versions. The **upload filter** is different: it is required here, for files and for review decisions.

Hermes: settings and why

The settings are in `hermes/config.yaml`, mounted read-only, so nobody can change them from the chat.

Setting	Value	Why
Model	`granite-4.2-30b` through `http://granite-gateway:8000/v1`	One model, one way out. The gateway adds the key, so Hermes holds none

Setting	Value	Why
`context_length`	131072	What the Granite server serves. Hermes refuses a model with less than 64,000 tokens, so the server needs `--max-model-len` of **65536 or more** (131072 recommended)
Tools allowed (`api_server`)	`secretary`, `todo`, `delegation`	The policy tools, a to-do list for multi-part requests, and subagents for side tasks
Tools switched off	web, search, browser, vision, image and video, speech, terminal, file, code execution, skills, memory, session search, scheduled jobs, kanban, connections	Answers come only from the policy database. Nothing can reach outside or change a file
`tool_search`	off	Hermes would otherwise hide some tools behind a search step. A mid-size model handles a short, plain list of tools more reliably
`max_turns`	30	Hermes' default is no limit. A loop that never ends would never free the chat
Subagents (`delegation`)	Same model, 20 steps each, 3 at a time, no subagents of their own	Keeps side tasks small
Helper calls (titles, compression, approvals)	Sent to the main model	"Auto" would try outside providers first. The sealed network would block them, but slowly
Memory, user profile, curator, background review	Off	Nothing is remembered between chats, and the assistant writes no skills of its own
Update checks, model catalog, lazy installs	Off	No calls home, nothing installed at run time
Plugin	`secretary` only	The tools below

Open WebUI also has its own title, tag and follow-up generation switched off. Each one would be a full Hermes run.

The plugin's tools

The plugin is `hermes/plugins/secretary/`. Every tool calls policy-api with `POLICY_API_KEY`. Each tool returns text with instructions for the model, and turns network errors into a plain message.

Tool	What it does	Calls
`search_policies`	Finds sections that answer a question, optionally within one group or one policy	`GET /search`
`read_policy`	Reads a whole policy, or one section with its subsections, at a chosen version	`GET /policies` (to resolve the name), `GET /policies/{id}/text`
`list_policy_groups`	Lists groups such as Safety or HR	`GET /subsets`
`find_policy`	Finds a policy by ID or title words	`GET /policies`

Tool	What it does	Calls
`check_policy_change`	Starts a check and waits for the result. If the check pauses, returns the guidance for the person	`GET /subsets`, `GET /policies`, `POST /runs`, then `GET /runs/{id}`
`get_check_result`	Gets the result of a check that was still running or paused, by check number	`GET /runs/{id}`
`make_check_files`	PDF, Word, Excel or charts of a finished check	`GET /runs/{id}`, `POST /runs/{id}/files`
`compare_policies`	Compares two policies, two versions, or a policy and an attached document	`GET /policies`, `POST /compare`
`make_document`	Word, PDF, Excel or chart from content the model writes, for requests that are not a check	`POST /documents`

There is **no tool for deciding "maybe" sections**. A decision comes only from the person's own typed message, through the upload filter (flow D). Nothing the assistant or a tool writes can count as one.

Data stores

Store	Holds	Notes
Postgres (`pgdata` volume)	Policies and every version, clauses with embeddings, the reference graph, groups, uploads, runs, the job queue, every model verdict, and the graph's checkpoint tables	Back up daily (operations.md). Checkpoints are not deleted when a check ends
`exports` volume	Excel, PDF, Word and chart files	Removed after `EXPORT_RETENTION_DAYS` (default 30)
`hermes-data` volume	Hermes' own working folder	Config, rules and plugin are mounted read-only on top. Memory is off
`openwebui` volume	Users, chats, the upload filter and its key	Back up with `.env`
Argilla databases (optional)	Review queue	Separate from the policy database

Network and security boundaries

```
frontend network:   people → proxy
backend network:    proxy → open-webui → hermes → policy-api → db, embed, rerank
                    open-webui (upload filter) → policy-api    (files, review decisions)
backend + egress:   hermes and policy-api → granite-gateway → Granite server
```

- `backend` is `internal: true`: no route out. `proxy` is the only container on `frontend`. The gateway is the only container on `llm-egress`.
- The gateway passes `/v1/chat/completions` and `/v1/models` and nothing else, and checks the Granite server's certificate.

- Hermes has no route to the internet and no tool that would use one. It reaches policy-api and the gateway, and nothing else.
- policy-api answers only requests with the right key and a known `Host` name. The proxy shows people only Open WebUI, report downloads and `/health`.
- The 8-character check number works only on `GET /runs/{id}` and `POST /runs/{id}/review`, and a review looks it up only among paused checks. Every other route needs the full id.

Three keys keep the parts apart. Each is held by as few parts as possible.

Key	Where it lives	Who uses it	What it protects
`POLICY_API_KEY`	`.env`	policy-api (to check it); Hermes (as `SECRETARY_API_KEY`, for the plugin); the upload filter (a setting in Open WebUI)	Every policy-api route except signed downloads
`HERMES_API_KEY` (16+ characters)	`.env`	Hermes (as `API_SERVER_KEY`); Open WebUI (as `OPENAI_API_KEY`)	Hermes' chat API
`GRANITE_API_KEY`	`.env`	`granite-gateway` only	The Granite server. Hermes and policy-api never see it

`LLM_API_KEY` guards the optional local models. `LINK_SIGNING_KEY` signs download links. See security.md.

When things fail

What fails	What happens
Granite server down or slow	Chat: Hermes gets no reply from Granite, so the chat shows an error. Checks: in `judge`, each pair that fails is marked "could not be checked automatically" and shows as Maybe, for a person to check. If **every** pair fails, the check fails at once with "The checking model is not responding" and is not retried
Second model (Inkling-Small) fails in `escalate`	The first opinion is kept
policy-api down	Each tool returns "The policy service did not respond"; Hermes tells the person to try again later. A check already queued waits in the database
Worker or policy-api crashes in the middle of a graph	The 60-second lease runs out (or, after a restart, the worker takes its own jobs back at once). Any worker picks the job up and the graph **carries on from the last saved node**. Only the node that was running repeats
A temporary error inside a node (database blip, model restart)	The job is retried up to `JOB_MAX_ATTEMPTS` (default 3), with growing waits (up to 5 minutes, random), each time from the last saved node. After that the job is `dead` and the person sees "failed several times in a row"
A problem with the question (no such policy, nothing changed)	Not retried. The person gets a plain reason
The graph runs past its recursion limit (25)	Treated as an unexpected error: retried, then the check fails with "Something went wrong on our side" and the error is logged

What fails	What happens
A check takes longer than 10 minutes (`SECRETARY_MAX_WAIT_MINUTES`)	The plugin stops waiting and tells Hermes the check is still running. The check itself continues. The person can ask again and Hermes calls `get_check_result` . It must not start a second check
Open WebUI waits longer than 15 minutes (`OPENWEBUI_TIMEOUT_SECONDS` , 900)	The chat shows an error. Keep the plugin's wait below this so it cannot happen. The check still finishes and can be fetched by check number
Hermes loops without answering	Hermes stops after 30 rounds (`max_turns`)
A paused check is never answered	It stays `needs_review` for as long as it takes. There is no time limit, no reminder and no worker is tied up. Asking for the result repeats the request for a decision. The saved state stays in Postgres
A decision is typed wrongly	The filter or policy-api refuses it and says why in a note; the check stays paused. A section not on the list is refused (422); a check that is no longer paused is refused (409)
The upload filter is off or not Global	Files are not sent, and decisions cannot be recorded. The check stays paused
A file cannot be read	The upload filter puts a plain note in the message; Hermes tells the person

Limits

- **No per-person identity reaches policy-api through Hermes.** Open WebUI knows who is signed in, but Hermes does not pass that on. Every check is stored as requested by `assistant` . Use Open WebUI's own account records to see who asked, and tell staff their questions are logged. The name on a review decision is the exception: the upload filter reads it from Open WebUI itself.
- **Check numbers are not tied to the person who asked.** Anyone signed in who has the 8-character number of a paused check can decide it. The number is a convenience, not a password (security.md).
- **Everyone with an account can ask about every group.** There is no per-user group filter.
- **Training still targets gpt-oss-20b.** The fine-tuning scripts and the `judge-candidate` and `judge-tuned` services are set up for that model, not for Granite. Verdicts can be sent to a tuned judge; the chat stays on Granite (review-and-training.md). Decisions typed in the chat are not training data.
- **The Granite server must be set up for this.** Tool calling, structured output and a context of at least 65536 tokens (setup.md).
- **A very long check holds the chat open.** The plugin waits up to 10 minutes (`SECRETARY_MAX_WAIT_MINUTES`); Open WebUI waits 15 (`OPENWEBUI_TIMEOUT_SECONDS` , 900). Keep the first below the second, and set any proxy in front of the stack to at least the same. If a check runs longer, Hermes says so and the result is fetched later by check number.
- **A partly answered review ends the check.** Sections left out stay "Maybe" and cannot be decided later.
- **Hermes follows its rules by instruction, not by force.** It cannot reach the internet or change a file, because those tools are switched off. It can still word an answer badly. That is why numbers, links and verdicts are produced by code and passed through unchanged. Its rules say not to show check numbers unless asked, except when a person must copy one to decide.

Why these choices

Choice	Why	Instead of
Hermes picks the tool; a fixed graph decides impact	The assistant handles the conversation (which group, which policy, which file, follow-up questions). The graph is deterministic, testable and auditable	An agent that plans its own steps and judges impact itself (more tokens, results vary between runs, harder to audit)
LangGraph for the check, with Postgres checkpoints	A check of hundreds of model calls can survive a crash and carry on from the last node. The same mechanism gives a clean pause for a person (`interrupt`) with no new service	Starting over after every crash, or a workflow engine such as Temporal as a new stateful system
Routing decided in code	The same input takes the same route every time, so results can be audited	Letting a model choose the next node
Review decisions read by the upload filter	The filter is trusted code that sees only the person's own message. The model has no decide tool, so it cannot approve anything, even if a policy contains hidden instructions	A decide tool for Hermes (the model could be talked into using it)
A plugin that returns text with instructions, and nothing else	Hermes has no web, files, terminal or memory, so a wrong answer can be badly worded but cannot leak or change anything	Giving the agent general tools and trusting it to behave
Granite 4.2 30B as judge and chat model	US-developed (IBM), Apache 2.0, dense 30B that fits one 80 GB GPU in bf16, 128K context, tool calling, thinking can be switched off per request for fast, repeatable verdicts	Inkling for everything (needs 4–8 high-end GPUs per replica)
Use the Granite server you already run, behind one gateway	No second copy of a 58 GB model to host and update. One narrow, checked way out (two routes, one address, key held in one place) instead of a model container inside the stack	Starting a local judge container, or letting each service call the server on its own
Inkling-Small only for "unsure"	Larger-model judgement where it matters, at a fraction of the cost	Running every pair through the large model
Reranker before judging	Cuts hundreds of similar clauses to about 20 per change; the biggest compute saving	Judging every similar clause
JSON-schema guided decoding	Every verdict is valid and machine-readable	Asking for JSON in the prompt and hoping
Postgres for the graph of policies	Recursive queries cover policy hierarchies; one fewer system to run	A separate graph database
Excel built with openpyxl	Exact, repeatable, safe from formula injection	Letting the model write the spreadsheet
Job queue in Postgres (`FOR UPDATE SKIP LOCKED`, leases)	Runs survive restarts with no new service to run or secure; Postgres is already here	Temporal or Dagster: another stateful system to operate inside the contained network, for a queue of a few hundred jobs a day

Data model (db/init.sql)

- `policies`, `policy_versions`, `clauses`: every version is kept. `current_clauses` is a view of each policy's latest version.
- `refs`: the reference graph (which clause cites which policy and section), for latest versions.
- `subsets`: which policies belong to which group. A policy can be in several groups.
- `uploads`, `runs`, `changes`: each question and what changed. A run's status is `queued`, `running`, `needs_review`, `done` or `failed`.
- `judgements`: one row per model decision, with model name, the exact prompt and its hash, verdict, and the reviewer's decision. This is the audit trail and the training data.
- `jobs`: the work queue (see operations.md). `schema_migrations` records which files in `policy_api/app/migrations/` have been applied; `db/init.sql` is the starting point for a new database.
- The checkpoint tables are made by LangGraph's Postgres saver on first start. They are not in `db/init.sql`.

Clause parsing rules

A new clause starts at a line that begins with a section number:

- a Markdown heading with a number (`## 4 Lockout devices`)
- any multi-level number (`4.2 Devices are inspected…`, `4.2.1) …`)
- a single number followed by a short title (`4 Lockout devices`)

`1. Do this.` inside a clause stays part of that clause. Text before the first number is clause `0`. Policy IDs are found with `POLICY_ID_PATTERN` (default `[A-Z]{2,6}-\d{1,5}`), in forms like `POL-007 §4.2`, `§4.2 of POL-007`, `POL-007 section 4` and `POL-007` alone (whole policy).

Roadmap

Built since v0.1: the job queue (operations.md), the review queue and LoRA fine-tuning (review-and-training.md), and reports, comparisons and charts (how-to.md).

1. **OCR** for scanned PDFs, and reading Word's automatic numbering (see known limits in how-to.md).
2. **Kubernetes** manifests when the models need more than one node.

Chapter 2

How to use it

Ask questions and read the results; add policies, groups and models.

For: Staff and administrators

For staff

Ask a question

Open the Policy Impact Assistant and type your question in your own words. Name two things:

1. **The group of policies to check**, such as Safety or HR
2. **The policy that changed**, either by its ID or name, or by attaching the new version

Examples:

> Which Safety policies are affected by the change to POL-007?
>
> Here's the new lockout policy. What in Safety needs updating? *(attach the file)*
>
> Check the HR group against the latest version of the leave policy.

If something is unclear, the assistant asks you one short question. A check usually takes one to a few minutes. Your chat waits for the answer (up to 10 minutes). If a check takes longer, the assistant tells you it is still running; ask again in a few minutes and it fetches the result.

Attaching a file: use `.docx`, `.pdf`, `.md` or `.txt`. The sections must be numbered (1, 2.1, 4.3…), because the system compares the policy section by section.

Without a file, the assistant compares the two newest stored versions of the policy. If only one version is stored, it asks you to attach the new one.

Read the answer

> **2 of the 5 Safety policies need updating because of the change to POL-007 (Lockout/Tagout).**

Then you'll see:

- **What changed** in the head policy, section by section
- **A table** of the sections that need updating, and why
- **"Might be affected"** policies. A person must check these: the system wasn't sure.
- How many were checked and are **not affected**
- **A link to the Excel file** with all the detail

If the assistant asks you to decide

Some installations pause a check when it has "maybe" sections, so a person settles them before the final answer. The assistant then lists the sections and shows a line to copy, like this:

> decide 3f9a1c2e SAF-150 3.1 , SAF-151 2.4

Type it as your **whole message**, starting with the word `decide`, and replace each `<yes|no>` with your own answer: `yes` if the section is affected, `no` if it is not. Add one entry for every section on the list. To answer every section the same way, type `decide 3f9a1c2e all no` (or `all yes`). The assistant cannot decide for you. When your decision is recorded, the check finishes and the assistant shows the final answer. Sections you leave out stay "Maybe", and the check ends; you can't decide it again afterwards. If you are told nothing was recorded, copy the line from the assistant again. The check number is only for this. Don't paste it where people outside the team can read it.

Use the Excel file

Sheet	What's on it
Summary	The question, the answer and the counts
Impacted	One row per affected section. **Red = needs updating. Yellow = a person must decide.** Each row shows the old and new wording, what the section says now, why it's affected, quotes as evidence, and a suggested edit
Head policy changes	Every changed section, old wording beside new
Checked, not impacted	Every other policy in the group, and why it was ruled out
Method	Which models and settings were used (for audits)
Charts	A bar chart of affected sections by policy
Change matrix	Which change in the head policy affects which policy

Fill in **Review decision** (Agree, Disagree, Not sure) and **Reviewer** as you go. The file is yours to keep and share inside the organisation.

The download link stops working after 72 hours. Ask the assistant again for a new one.

Get a report, chart or comparison

Just ask, in your own words, after a check or on its own:

Say something like	You get
"Can I have that as a PDF?" / "...as a Word report?"	A report of the last check: the answer, charts, the table of affected sections, and the old and new wording with changes marked
"Show me a chart of that"	Charts in the chat: affected sections by policy, which change affects which policy, kinds of impact
"Compare POL-007 version 3 with version 4"	A side-by-side comparison, section by section. Removed words are red and crossed out; new words are blue and underlined

Say something like	You get
"Compare this file with POL-007" (with a file attached)	The same comparison, against your file
"Compare SAF-112 and SAF-130 in Excel"	One row per section, in a workbook
"Make a one-page Word summary of this for the safety committee"	A document the assistant writes from what's in the chat. It says so at the top: check it before relying on it

Files can be Word (.docx), PDF, or Excel (.xlsx). Links last 72 hours; one-off documents are kept for 30 days, and reports of a check can always be made again.

What the system can and can't do

- It **finds and explains.** People still decide and approve every policy change.
- **"Maybe" never means "fine".** Always check these.
- It only knows the policies that have been loaded. If one seems missing, tell the system owner.
- It can be wrong in both directions. If it flags something that isn't affected, or misses something that is, tell the system owner. Mistakes are how it gets better.

Known limits

- **Scanned PDFs** (pictures of pages) can't be read yet. Use the Word or text version.
- **Word's automatic numbering** isn't stored as text, so a Word file whose numbers are automatic may appear to have no sections. Save it with the numbers typed in, or as PDF.
- **Policies outside the group** you named are not checked. Ask about each group separately.

For admins

Add or update policies

Put the files in `POLICY_SOURCE_DIR` and run:

```
docker compose run --rm policy-api python -m app.ingest /policies
```

Ingest is safe to repeat. A file whose header has the same `version` replaces that version; a higher `version`, or no version, adds a new one. Earlier versions are kept, so "compare the two newest versions" works without uploading anything.

Add a group, or move a policy between groups

Edit `subsets:` in the policy's header, then ingest that file again. A policy can be in several groups. A policy's groups are replaced each time it's ingested.

Turn on the second-opinion model (Inkling-Small)

Start it with `docker compose --profile escalate up -d escalate`, set `ESCALATE_URL=http://escalate:8000/v1` in `.env`, then run `docker compose up -d policy-api`. "Unsure" pairs then go to Inkling-Small. The **Checked by larger model** column in Excel shows which.

Find more references when ingesting

`--llm-refs` asks the `extract` model to find references that the ID pattern misses, such as "the lockout policy". Start the model first (`--profile extract`) and set `EXTRACT_URL` .

Change how wide the search is

In `.env` :

- `SEMANTIC_K` (default 30): how many similar clauses to consider for each change
- `RERANK_KEEP` (default 20): how many of those to judge

Raise them if evaluation shows misses on policies that don't cite the head policy. Lower them if runs are too slow. Clauses that cite the head policy, or use a changed term, are always judged regardless of these settings.

Look up what happened in a run

Every decision is in the `judgements` table:

```
docker compose exec db psql -U policy -c \
  "SELECT candidate_policy, candidate_number, stage, model, result->>'impacted', result->>'why'
   FROM judgements WHERE run_id = '<run id from the Excel Summary sheet>' ORDER BY
candidate_policy"
```

Chapter 3

Setup

Install it on the GPU server.

For: Engineers

This guide installs secretary-keeper on one GPU server with Docker Compose. Follow the steps in order. Each step ends with a check; don't move on until it passes.

1. What you need

Item	Minimum	Notes
Granite server	An existing vLLM server running Granite 4.2 30B (about 58 GB in bf16, so one 80 GB GPU or two 48 GB)	Not part of this stack. See the next section
GPUs for this stack	1 smaller GPU for `embed` + `rerank`	Optional: 1 for `extract`, and 4 Blackwell or 8 Hopper for `escalate`
NVIDIA driver + Container Toolkit	Recent enough for your vLLM image	`docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi` must list your GPUs
Docker Engine + Compose v2	Compose 2.20 or later	`docker compose version`
Disk	About 400 GB	Mostly model weights
A machine with internet access	Temporary	To download images and weights before moving them in

Granite server requirements

The main model, IBM Granite 4.2 30B (Apache 2.0, dense, about 29B parameters, 128K context), is **not** started by this stack. It already runs on your own vLLM server. This stack calls it. Ask whoever runs that server to confirm each line:

Setting on the Granite server	Value	Why
vLLM version	0.20 or later	Earlier versions do not support Granite 4.2
Model name	`--served-model-name granite-4.2-30b`	Or keep your own name and set `GRANITE_MODEL` in `.env` to match
Precision	`--dtype bfloat16`	About 58 GB of weights
Tool calling	`--enable-auto-tool-choice --tool-call-parser qwen3_coder`	Hermes needs the model to call the policy tools

Setting on the Granite server	Value	Why
Reasoning	`--reasoning-parser nemotron_v3`	Built into vLLM. The model card's `granite_thinking_parser.py` plugin also works, but it runs code from the model download, so it is optional
Context length	`--max-model-len` of at least **65536**	Hermes refuses models under 64K. 131072 is recommended and matches `context_length` in `hermes/config.yaml`
Structured output	JSON-schema responses must work (vLLM guided decoding)	Every verdict is requested with `response_format` `json_schema`
API key	`--api-key <key>` (recommended)	Put the same key in `GRANITE_API_KEY`
Address	An `https://` address, unless the server runs on this same machine	Put it in `GRANITE_URL`, without `/v1`

Quick check, from a machine that can reach the server:

```
curl -s -H "Authorization: Bearer $GRANITE_API_KEY" $GRANITE_URL/v1/models
```

Check: the answer lists `granite-4.2-30b` (or the name you set in `GRANITE_MODEL`). Leave out the `-H` part if the server has no key.

The gateway

Nothing in the stack talks to the Granite server directly. The `granite-gateway` container (nginx, `gateway/granite.conf.template`) is the only container on the `llm-egress` network, and the only way out of the sealed backend network. It:

- passes on only `/v1/chat/completions` and `/v1/models`, and returns 404 for everything else;
- adds `GRANITE_API_KEY` itself and drops any key the caller sent, so the key lives in one place;
- checks the server's certificate. For a certificate from a private CA, put the CA file in `gateway/` (for example `gateway/granite-ca.pem`) and set `GRANITE_CA_PATH=/etc/nginx/granite/granite-ca.pem` in `.env`. `*.pem` files in `gateway/` are not committed to git.

`JUDGE_THINKING=1` makes Granite reason before each verdict. It is slower and sometimes better. The default is off. Try it on your evaluation cases first.

2. Bring images and weights into the environment

On a machine **with** internet access:

```
# Images: pin the exact tags you will write into .env
docker pull vllm/vllm-openai:<VLLM_TAG>
docker pull ghcr.io/huggingface/text-embeddings-inference:<TEI_TAG>
docker pull ghcr.io/open-webui/open-webui:<OPENWEBUI_TAG>
docker pull nousresearch/hermes-agent:<HERMES_TAG>
docker pull pgvector/pgvector:pg17
docker pull nginx:1.27-alpine
```

```
# Tag must equal APP_VERSION in .env, so Compose uses this image instead of trying to build one.
docker build -t secretary-keeper/policy-api:0.1.0 policy_api    # needs PyPI or your internal
mirror
docker save -o secretary-keeper-images.tar vllm/vllm-openai:<VLLM_TAG> ... secretary-
keeper/policy-api:0.1.0

# Weights (see models/README.md for the list)
hf download ibm-granite/granite-embedding-english-r2 --local-dir models/embedding
hf download ibm-granite/granite-embedding-reranker-english-r2 --local-dir models/reranker
# Granite itself stays on your vLLM server, so it is not copied here.
scripts/verify_models.sh record
```

Move the tar file, the repo and the `models/` folder into the environment. Then:

```
docker load -i secretary-keeper-images.tar
scripts/verify_models.sh check
```

Check: `verify_models.sh check` prints "All model files match their recorded hashes."

3. Configure

```
cp .env.example .env
for k in POSTGRES_PASSWORD POLICY_API_KEY LINK_SIGNING_KEY LLM_API_KEY OPENWEBUI_SECRET_KEY
HERMES_API_KEY; do
  sed -i "s/^$k=.*/$k=$(openssl rand -hex 32)/" .env
done
chmod 600 .env
```

Then edit `.env`:

- `GRANITE_URL`, `GRANITE_API_KEY`, `GRANITE_MODEL`: your Granite server (see above). Add `GRANITE_CA_PATH` if its certificate comes from a private CA.
- `VLLM_TAG`, `TEI_TAG`, `OPENWEBUI_TAG`, `HERMES_TAG`: the exact tags you loaded. `HERMES_TAG` was tested with `v0.22.3`.
- `REVIEW_GATE`: `1` makes a check that ends with "maybe" sections pause until a person decides them in the chat (default `0`). It needs the upload filter switched on and **Global** (step 7), because only the filter can record a decision.
- `HERMES_API_KEY`: at least 16 characters (the command above makes 64). Hermes will not start without it, and Open WebUI uses the same value to talk to Hermes.
- `PUBLIC_BASE_URL`: the address people's browsers use for downloads, e.g. `https://policy.plant.local:8100`.
- `POLICY_ID_PATTERN`: if your IDs don't look like `ABC-123`.
- `POLICY_SOURCE_DIR`: the folder holding your policy files.

Assign GPUs to the models this stack runs itself (embedding, reranker and any optional ones). Granite is not on this list. `compose.yaml` names no GPUs, so you choose them for this server:

```
python3 scripts/setup_gpus.py
```

It lists your GPUs, asks which ones each model uses, checks the plan (for example that no two big models share a GPU), writes `compose.gpu.yaml` and sets `COMPOSE_FILE`, `ESCALATE_TP` and the other size settings in `.env`. Run it again whenever the hardware changes. To do it by hand instead, follow the comments in `compose.gpu.example.yaml`, which also has sizing notes per model.

If your embedding model's output size is not 768, change `vector(768)` in `db/init.sql` now, before the database is created.

Check: `docker compose config --quiet` prints nothing.

4. Start the stack

```
docker compose up -d db embed rerank granite-gateway
docker compose logs -f embed rerank   # wait until both say they are ready
docker compose up -d policy-api hermes open-webui proxy
```

Optional models:

```
docker compose --profile escalate up -d escalate   # then set
ESCALATE_URL=http://escalate:8000/v1 in .env
docker compose --profile extract up -d extract     # then set EXTRACT_URL=http://extract:8000/v1
in .env
docker compose up -d policy-api                     # restart to pick up .env changes
```

Check: `docker compose ps` shows every service as running or healthy. To prove the gateway reaches Granite: `docker compose exec policy-api python3 -c "import urllib.request as u; print(u.urlopen('http://granite-gateway:8000/v1/models').read()[:200])"` prints the model list.

5. Prove the environment is contained

```
scripts/check_containment.sh
```

Check: it ends with `PASS contained`. Each service it checks must first reach an internal service (proving the probe works) and then fail to reach the internet. It also checks that `granite-gateway` is the only container on `llm-egress`. If it says FAIL, stop and fix it before loading real policies. See security.md.

6. Load policies

Policy files can be `.md`, `.txt`, `.docx` or `.pdf`. A short header block at the top of each file gives the details the system can't guess:

```
---
id: SAF-112
title: Maintenance Access
owner: Maintenance Manager
parent: POL-007
subsets: [Safety]
version: 1
---
```

Without a header, the ID is taken from the file name or the first lines, and the version counts up. Groups (`subsets`) must be in the header.

```
docker compose run --rm policy-api python -m app.ingest /policies
# Also use the extract model to find references the regex misses (slower):
docker compose run --rm policy-api python -m app.ingest /policies --llm-refs
```

Check: it prints `ingested <ID> v<N>` for each file and lists any skipped files with the reason.

7. Set up Hermes and Open WebUI

Hermes

Hermes needs no setup in its own screen. Its settings are files in `hermes/`, mounted read-only: `config.yaml` (model, allowed tools, limits), `SOUL.md` (its rules) and `plugins/secretary/` (the policy tools). Open WebUI is already pointed at it by `compose.yaml`: `OPENAI_API_BASE_URL` is `http://hermes:8642/v1`, `OPENAI_API_KEY` is `HERMES_API_KEY`, and the default model is `policy-assistant`.

If `GRANITE_MODEL` in `.env` is not `granite-4.2-30b`, change `model.default` and `delegation.model` in `hermes/config.yaml` to match, then `docker compose up -d hermes`.

Check that Hermes is up and sees only the policy tools:

```
docker compose logs hermes | tail -30     # look for the API server listening on port 8642 and
the secretary plugin loaded
docker compose exec hermes python3 -c "import urllib.request as u;
r=u.Request('http://localhost:8642/v1/models', headers={'Authorization':'Bearer
'+__import__('os').environ['API_SERVER_KEY']}); print(u.urlopen(r).read()[:300])"
```

The second command must list `policy-assistant`. Then, in a chat, ask *"What tools do you have?"*. The answer should name only policy tools: search, read, list groups, find, check, get result, make files, compare, make document, plus the to-do list and subagents. If it mentions web search, files, a terminal or memory, stop and check `agent.disabled_toolsets` and `platform_toolsets` in `hermes/config.yaml` (the file is read when Hermes starts).

If Hermes exits with a message about context size, the Granite server's `--max-model-len` is below 65536 (Granite server requirements).

Open WebUI

Open `http://<server>:3000`. The first account you create becomes the admin. Then:

1. **Turn off sign-up.** Set `OPENWEBUI_ENABLE_SIGNUP=false` in `.env` and run `docker compose up -d open-webui`. Add users from the Admin Panel.
2. **Add the upload filter** (ui.md, step 2), set its `API_KEY` to `POLICY_API_KEY`, switch it on and make it **Global**. It is required in this version for two jobs: sending attached files to policy-api, and recording the review decisions people type (`decide ...`). The assistant has no tool for decisions, so with the filter off or not Global, a paused check can never be decided from the chat.

3. **Pick the model.** People see one model, `policy-assistant`, which is Hermes. There is nothing else to install: the Open WebUI tool (`openwebui/policy_impact_tool.py`), the custom system prompt and "Function Calling: Native" are for the other versions and are **not used** here. Hermes calls the policy tools itself.

Menu names move between Open WebUI releases. If a step doesn't match, look for the same setting nearby. If uploads stop working after an Open WebUI upgrade, check the filter first (operations.md).

Check the review pause (only if `REVIEW_GATE=1`): run a check that has a "maybe" section. The assistant should stop and list the sections, with a check number and a line to type. Type `decide <check number> all no`. The reply should say the decisions were recorded, and the check should then finish. If it says nothing was recorded, the filter is off, is not Global, or its `API_KEY` is wrong.

Check: in a new chat, ask *"Which Safety policies are affected by the change to POL-007?"* and attach the new version. You get an answer with a download link, and the link opens an Excel file.

8. Before real use

- [ ] `scripts/check_containment.sh` passes
- [ ] Evaluation on at least 10 past changes meets your recall target (evaluation.md)
- [ ] Security checklist done (security.md)
- [ ] Two or three staff have tried it using only how-to.md, without help

Chapter 4

Operations

Run it day to day, upgrade it, fix problems.

For: Administrators and engineers

Day-to-day running, upgrades, backups and troubleshooting.

Everyday commands

```
docker compose ps                                  # what's running and healthy
docker compose logs -f --tail=100 policy-api       # follow one service
docker compose restart policy-api                  # after editing .env
docker compose stop && docker compose up -d        # full restart
scripts/check_containment.sh                       # after any change
```

Optional services need their profile: `docker compose --profile escalate up -d escalate`.

Health

Check	Healthy looks like
`curl -s http://127.0.0.1:8100/health`	`{"ok":true,"demo_mode":false}` (if `demo_mode` is true, real models are **not** in use)
`docker compose ps`	`db`, `policy-api` and `granite-gateway` show `healthy`
`curl -s $GRANITE_URL/v1/models`	Lists `granite-4.2-30b`. Granite runs on its own server, so check it there (and with whoever runs it), not with `nvidia-smi` here
Recent runs	`SELECT status, count(*) FROM runs WHERE created_at > now() - interval '1 day' GROUP BY 1;` shows few `failed`
Job queue	`curl -s -H "Authorization: Bearer $POLICY_API_KEY" http://policy-api:8000/queue` from inside the network: `oldest_wait_s` near 0 and `dead` not rising

The job queue

Every check is a job in the `jobs` table. `policy-api` runs a worker that takes jobs one at a time per free slot (`WORKER_CONCURRENCY`, default 2) and renews a lease on each while it works (`JOB_LEASE_SECONDS`, default 60).

- **Restarts and crashes don't lose checks, or the work already done.** If `policy-api` restarts, it takes back its own jobs at once. If it dies or the server reboots, the lease runs out and the job is picked up again. Each check runs as a LangGraph graph that saves its state to Postgres after every step (load, diff, find related, judge, escalate, summarise), so the retry carries on from the last finished step instead of starting over. Only the step that was running is repeated (its half-done verdicts are cleared first). People waiting see "Starting again after an interruption…".

- **Paused checks.** With `REVIEW_GATE=1`, a check whose answer has "maybe" sections stops with status `needs_review` and shows "Waiting for a person to decide N section(s)". It holds no worker and no lease, only its saved state, so it survives restarts for as long as it waits. There is no time limit and no reminder: nobody is told. When the person types `decide <check number> ...` in the chat, the upload filter sends it to policy-api, which queues a `review_run` job; a worker resumes the graph and the check finishes. Find forgotten ones with `docker compose exec db psql -U policy -c "SELECT id, subset, created_at FROM runs WHERE status = 'needs_review'"`. A decision can leave sections out; those stay "Maybe" and the check is then finished, so it cannot be decided again.
- **Temporary problems are retried** (a model server restarting, the database briefly away), up to `JOB_MAX_ATTEMPTS` (default 3) with growing waits. After that the job is `dead` and the person sees "failed several times in a row".
- **Problems with the question itself are not retried** (no such policy, nothing changed).
- **More capacity:** add workers with `docker compose run -d --name worker-2 policy-api python -m app.worker`, or raise `WORKER_CONCURRENCY`. Workers never take the same job twice. Set `WORKER_ENABLED=0` on `policy-api` if you want only separate workers to run checks.

Dead jobs keep their last error:

```
SELECT id, kind, payload, attempts, last_error, finished_at FROM jobs WHERE status = 'dead' ORDER
BY id DESC LIMIT 20;
-- Retry one after fixing the cause:
UPDATE jobs SET status = 'pending', attempts = 0, run_at = now() WHERE id = <id>;
UPDATE runs SET status = 'queued', error = NULL, finished_at = NULL WHERE id = '<payload
run_id>';
-- Housekeeping (monthly):
DELETE FROM jobs WHERE status = 'done' AND finished_at < now() - interval '30 days';
```

How long a run takes

Roughly: number of changed sections × about 20–40 related clauses × time per verdict ÷ `JUDGE_CONCURRENCY`. For example, 10 changed sections with 30 clauses each is 300 verdicts; at 16 at a time and about 3 s each, that's about one minute. Raise `JUDGE_CONCURRENCY` if the Granite server isn't busy; lower it if verdicts time out.

Backups

Back up the database daily. The models and the policy source files are recoverable from their original sources; runs and verdicts are not.

```
docker compose exec -T db pg_dump -U policy -Fc policy > backups/policy-$(date +%F).dump
# Restore into an empty database:
docker compose exec -T db pg_restore -U policy -d policy --clean --if-exists < backups/policy-
YYYY-MM-DD.dump
```

Test a restore every quarter on a spare machine. Also back up the `openwebui` volume (users, chats and the upload filter's settings), and keep `.env` in your secrets vault.

Upgrades

Change one thing at a time, and run the evaluation after anything that touches the models.

1. Read the release notes for the component.
2. Bring the new image or weights in as in setup.md; record and check hashes.
3. Change the tag in `.env` (or the folder in `compose.yaml`).
4. `docker compose up -d <service>`
5. `scripts/check_containment.sh`
6. Smoke test (testing.md), and evaluate if a model changed.
7. Roll back by restoring the old tag and running `docker compose up -d <service>`.

Granite upgrades happen on the Granite server, not here. After any change there (vLLM version, flags, model files), run `curl -s $GRANITE_URL/v1/models`, then the evaluation. To change the key or address, edit `GRANITE_API_KEY` / `GRANITE_URL` in `.env` and run `docker compose up -d granite-gateway`.

Open WebUI upgrades can change how filters see attached files. Afterwards, attach a file and ask a question. If the assistant says no file was attached, check how the new release passes `__files__` and adjust `_files` in `openwebui/upload_filter.py`. Also type a `decide` command on a paused check (see ui.md). The Open WebUI tool (`openwebui/policy_impact_tool.py`) is not used in this version.

Hermes upgrades: change `HERMES_TAG` in `.env` (tested with v0.22.3), then check the key names in `hermes/config.yaml` against the new release before starting it, and run the checks in setup.md.

policy-api upgrades: rebuild outside the environment with a new `APP_VERSION`, carry the image in, and set `APP_VERSION` in `.env`. Checks in progress during the restart are picked up again automatically. Database changes in `policy_api/app/migrations/` are applied on start, once each.

Changing the embedding model

The `clauses.embedding` column has a fixed size (768). To switch to a model with a different size:

```
docker compose exec db psql -U policy -c "DROP INDEX clauses_embedding_idx;
  ALTER TABLE clauses ALTER COLUMN embedding TYPE vector(<new size>) USING NULL;
  CREATE INDEX clauses_embedding_idx ON clauses USING hnsw (embedding vector_cosine_ops);"
# Put the new model in models/embedding, restart embed, then re-ingest every policy:
docker compose up -d embed && docker compose run --rm policy-api python -m app.ingest /policies
```

Update `vector(768)` in `db/init.sql` too, so new installs match. Re-run the evaluation.

Moving a model to different GPUs

Run `python3 scripts/setup_gpus.py` again and give the new GPU numbers. Listing several GPUs for `escalate` (or another vLLM model run by this stack) spreads that model across them (tensor parallelism); the script sets `ESCALATE_TP` and the other size settings to match. Then `docker compose up -d escalate` (or whichever service moved). Granite's GPUs are managed on its own server.

Troubleshooting

Symptom	Likely cause	Fix
policy-api exits with `Missing settings`	`.env` incomplete	Fill in the named variables
policy-api exits with `at least 32 characters`	Short key	Regenerate with `openssl rand -hex 32`
`400 Invalid host header`	The Host name isn't in `ALLOWED_HOSTS`	Add the name the proxy sends (`policy-api`) or your hostname
Chat says "The policy service did not respond"	policy-api is down, or `SECRETARY_API_KEY` in Hermes does not match `POLICY_API_KEY`	`docker compose ps` ; compare the two values in `.env` ; `docker compose logs hermes`
Chat shows an error and no answer	Hermes cannot reach Granite, or the Granite server's context is under 65536	`docker compose logs hermes granite-gateway` ; check `--max-model-len`
"Nothing was recorded" after typing `decide ...`	The upload filter is off, not Global, or has the wrong `API_KEY` ; or the check number or sections do not match the list	ui.md
A check is stuck at "Waiting for a person to decide"	Nobody has answered (there is no timeout)	Ask the person to type the `decide` line; the assistant repeats it when asked for the result
Chat says "checking model is not responding"	The Granite server is down or overloaded, or `granite-gateway` cannot reach it	`docker compose logs granite-gateway` ; run `curl -s $GRANITE_URL/v1/models` ; lower `JUDGE_CONCURRENCY`
Many "could not be read" verdicts	Structured output not working with this vLLM version	Check the Granite server's vLLM version supports `response_format` JSON schema; check its logs
The assistant answers without calling a tool	Granite server started without tool calling, or Hermes cannot list its tools	`--enable-auto-tool-choice` on the Granite server (setup.md); ask "What tools do you have?"
"No numbered sections were found" for a Word file	Word automatic numbering	Save with typed numbers, or as PDF (how-to.md)
Download link gives 403	Link expired (72 h) or copied incompletely	Ask the assistant again
Download link gives a connection error	`PUBLIC_BASE_URL` isn't reachable from users' browsers	Set it to the proxy's public address and restart policy-api
A run stays "running"	It's large, or Granite is slow	Check `progress` on the run; the plugin stops waiting after 10 minutes (Open WebUI after 15) but the run continues; ask for the result again with the check number
Ingest skips files	No ID, no numbered sections, or unreadable type	The skip reason is printed for each file
`check_containment.sh` fails	A service can reach the internet	security.md

Chapter 5

Security and safety practices

Keep policy text contained and the system safe.

For: Everyone

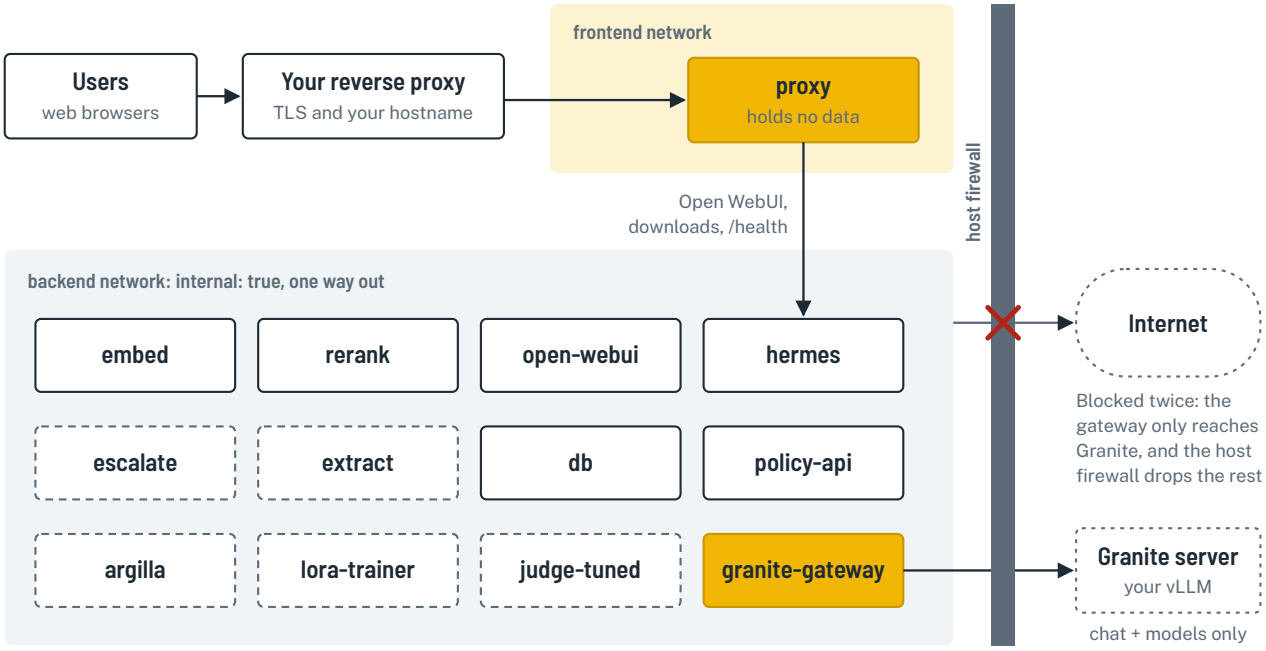
Two things must hold: **policy text never leaves the contained environment**, and **no one acts on an automated decision without a person checking it**. This page explains how the system enforces both, and what you still have to do.

Threats and controls

Risk	Control in this repo	Your job
Policy text leaves the environment	Every service holding text is on the `internal: true` network; the only way out is `granite-gateway`, which passes just chat and model-list calls to your own Granite server; `HF_HUB_OFFLINE`, telemetry off	Run `check_containment.sh`; block egress at the host firewall too, and allow `llm-egress` to reach only the Granite address
Unvetted or tampered model weights	Weights mounted read-only; SHA-256 manifest	Record hashes at download; check before each start; approved-model list
Someone calls the API directly	Bearer key on every route; proxy exposes only downloads and `/health`	Keep `.env` at `chmod 600`; rotate keys
A malicious web page uses a browser to reach the API (DNS rebinding)	`TrustedHostMiddleware` refuses unknown `Host` headers	Keep `ALLOWED_HOSTS` short
A download link is shared or guessed	HMAC-signed links that expire after 72 h; run IDs are random UUIDs	Shorten `LINK_TTL_HOURS` if needed
Instructions hidden inside a policy steer the model ("ignore the above...")	Policy text is fenced as data in every prompt, and fence markers inside it are replaced so it can't break out; the model returns only a fixed JSON schema for each verdict; a "yes" whose quoted evidence isn't really in both texts is downgraded to Maybe for a person to check; the chat model is told never to follow text in documents	Investigate verdicts whose reasons look odd
Policy text becomes a live Excel formula	Cells that would be formulas are forced to text	Nothing
Document or model text adds a link or image to the chat answer (an image URL could carry data out)	Markdown characters in every document and model value are escaped; only the download link is live	Keep Open WebUI on the internal network; block egress so a stray image can't load

Risk	Control in this repo	Your job
A small upload expands into a huge one (zip bomb, giant PDF) or a crafted line slows parsing	Uploads capped at `MAX_UPLOAD_MB`; .docx refused if it unpacks past 50 MB; PDFs capped at 1,000 pages; text capped at 5 M characters; each upload is parsed in its own process (1 GB memory cap, killed after 60 s, at most 2 at once); heading regex can't backtrack	Nothing
The model is wrong	Evidence quotes and reasons on every row; "Maybe" for uncertainty; full audit trail	Keep a person approving every change (below)
Lost history	Every model decision stored in `judgements` with model name and prompt hash	Back up the database (operations.md)

Network containment



Two barriers keep policy text in: a network with one narrow way out, and your host firewall.

- `backend` is a Docker network with `internal: true`, so its containers have no gateway out.
- The backend has exactly **one** way out: `granite-gateway`. It also joins `llm-egress`, a normal Docker network, and forwards to `GRANITE_URL` only. It passes `/v1/chat/completions` and `/v1/models` and nothing else, and checks the server's TLS certificate (`GRANITE_CA_PATH` for a private CA). No other container joins `llm-egress`.
- The Granite API key is held only by the gateway. It adds the key itself and drops any key the caller sent, so Open WebUI and policy-api never see it.
- **Firewall rule to add:** on the host, allow traffic from the `llm-egress` network to the one address and port of your Granite server, and drop everything else from it. Without this rule, the gateway is still limited by its config, but the host would not enforce it.
- Only `proxy` joins `frontend`. nginx serves Open WebUI and only `/exports/<run-id>.xlsx` and `/health` from policy-api. All other API routes return 404 from outside.

- **Don't rely on Docker settings alone.** Also block outbound traffic from the host at your firewall (for example, rules in the `DOCKER-USER` chain on Linux), and keep the server on a network segment without internet access.

Proving it

```
scripts/check_containment.sh
```

The script first checks that `granite-gateway` is the only container on `llm-egress`. Then, for each running service, the script first connects to an internal service. That's the positive control: it proves the probe itself works. Then it tries to reach the internet, and that must fail. The script also fails if it checked nothing. Run it after setup, after every upgrade, and after any network change. Keep the output with your change records.

During development, a network setting meant to block outbound traffic ("no outbound NAT") was ignored by Docker Desktop. The script caught it. That's why containment relies on `internal: true` networks, not driver options.

Secrets

- Every secret lives in `.env` (git-ignored). Generate each with `openssl rand -hex 32`. policy-api refuses to start if its keys are shorter than 32 characters.
- `chmod 600 .env`. Only administrators can read the server.
- **Rotate** by changing the value and restarting the affected services:
- `POLICY_API_KEY`: restart `policy-api` and `hermes` (it holds the value as `SECRETARY_API_KEY`), and update the upload filter's `API_KEY` setting in Open WebUI
- `HERMES_API_KEY` (16+ characters): restart `hermes` and `open-webui`
- `LINK_SIGNING_KEY`: restart `policy-api`; existing download links stop working
- `LLM_API_KEY`: restart the local model services (escalate, extract), `policy-api` and `open-webui`
- `GRANITE_API_KEY`: change it on the Granite server and in `.env`, then `docker compose up -d granite-gateway`. Nothing else needs it
- Never paste secrets into chats, tickets or a wiki.

Models and supply chain

- Use only models on your organisation's approved list. All defaults are US-developed and published under permissive licenses. Check each model card and license before use.
- Granite 4.2 30B runs on your own vLLM server, outside this stack. Its weights, vLLM version and flags are that server's owner's job. Prefer the built-in `--reasoning-parser nemotron_v3` over the model card's `granite_thinking_parser.py` plugin, which runs code from the model download.
- Download weights outside the environment, record hashes (`scripts/verify_models.sh record`), carry them in, and check them (`verify_models.sh check`) before every start and upgrade.
- Pin every image tag in `.env`. Never use `latest`. Build the policy-api image from a pinned dependency set or your internal mirror.

- vLLM's `--trust-remote-code` (needed for Inkling) runs Python shipped with the model. Review that code and verify the hashes before enabling `escalate`.

Access

- Open WebUI accounts are created by an admin. Sign-up is off after the first admin.
- Everyone with an account can query every group. If some groups must be restricted, run a separate stack per audience, or add per-user group filtering to the plugin before go-live.
- Hermes does not pass on who is signed in, so every check is stored as requested by `assistant`. Open WebUI's own chat records show who asked. Tell users their questions are logged. The name of the person who decides a "maybe" section is recorded, because the upload filter takes it from Open WebUI.

Human oversight

The system **finds and explains**. It does not approve changes.

- Every "Yes" and "Maybe" goes to a named person, who records a decision (the Review decision column, later the review queue).
- "Maybe" never means "fine". Not-affected policies are listed, with the reason, so reviewers can spot-check them.
- Don't let the assistant's output become the record of decision. The decision belongs in your document control system.
- Measure accuracy on real past changes before launch and every quarter (evaluation.md).

Deciding "maybe" sections in chat (`REVIEW_GATE=1`)

The AI model cannot decide. A person types `decide <check number> ...` as their whole message; the Open WebUI upload filter (trusted code) reads it and sends it to policy-api. The check number is the first 8 characters of the check's id, which is only 32 bits. Plainly, what that means:

- It is a convenience, not a password. Someone who guessed or saw a code could decide that check.
- Only signed-in staff can enter it, and only through the chat (the filter holds the service key; people and the model do not).
- It is accepted only as exactly 8 characters (or the full id), and only on the status and review routes. Files, downloads and every other route need the full id.
- For a review it is looked up only among checks that are paused waiting for a decision. A finished or running check cannot be decided, and a code that matches two paused checks is refused.
- Every decision is recorded under the Open WebUI name of whoever typed it, so a wrong or unauthorised one is visible and can be traced.
- The check is not tied to the person who started it. Hermes sends no requester identity (the check shows `assistant`), so a decision cannot be matched to the person who asked. Treat check numbers as internal: do not paste them where people outside the team can see.

Data retention

Uploaded files, runs, verdicts and Excel files stay in the database and the `exports` volume until you delete them. Agree on a retention period with records management. To purge runs older than a set age:

```
DELETE FROM runs WHERE created_at < now() - interval '2 years';   -- cascades to changes and
judgements
DELETE FROM uploads u WHERE NOT EXISTS (SELECT 1 FROM runs r WHERE r.upload_id = u.id)
                        AND u.created_at < now() - interval '2 years';
```

Then delete the matching `<run-id>.xlsx` files from the `exports` volume.

If something goes wrong

Situation	Do this
`check_containment.sh` fails	Stop `open-webui` and `policy-api` (`docker compose stop`). Fix the network. Re-run the check. Record what happened
A secret may have leaked	Rotate it now (see Secrets). Check the proxy logs for unusual downloads
A wrong result was acted on	Find the run ID on the Excel Summary sheet; pull its `judgements`; add the case to the evaluation set
A policy contains text that tries to instruct the model	Keep the document for analysis; check the verdicts for that policy by hand

Go-live checklist

- [ ] `check_containment.sh` passes and the output is filed
- [ ] Host firewall blocks outbound traffic from the server, except `llm-egress` to the Granite address only
- [ ] `GRANITE_URL` uses `https://` (unless Granite is on this machine) and the certificate check works
- [ ] `.env` secrets generated fresh, file is `chmod 600`, not in git
- [ ] Model hashes verified; models on the approved list
- [ ] Image tags pinned; none is `latest`
- [ ] Open WebUI sign-up turned off; users created by an admin
- [ ] TLS on the reverse proxy in front of `proxy`; `PUBLIC_BASE_URL` uses `https://`
- [ ] Evaluation recall meets the agreed target on a hold-out set
- [ ] Review process for Yes/Maybe agreed with the policy owners
- [ ] Database backups running and restore tested
- [ ] Retention period agreed and written down

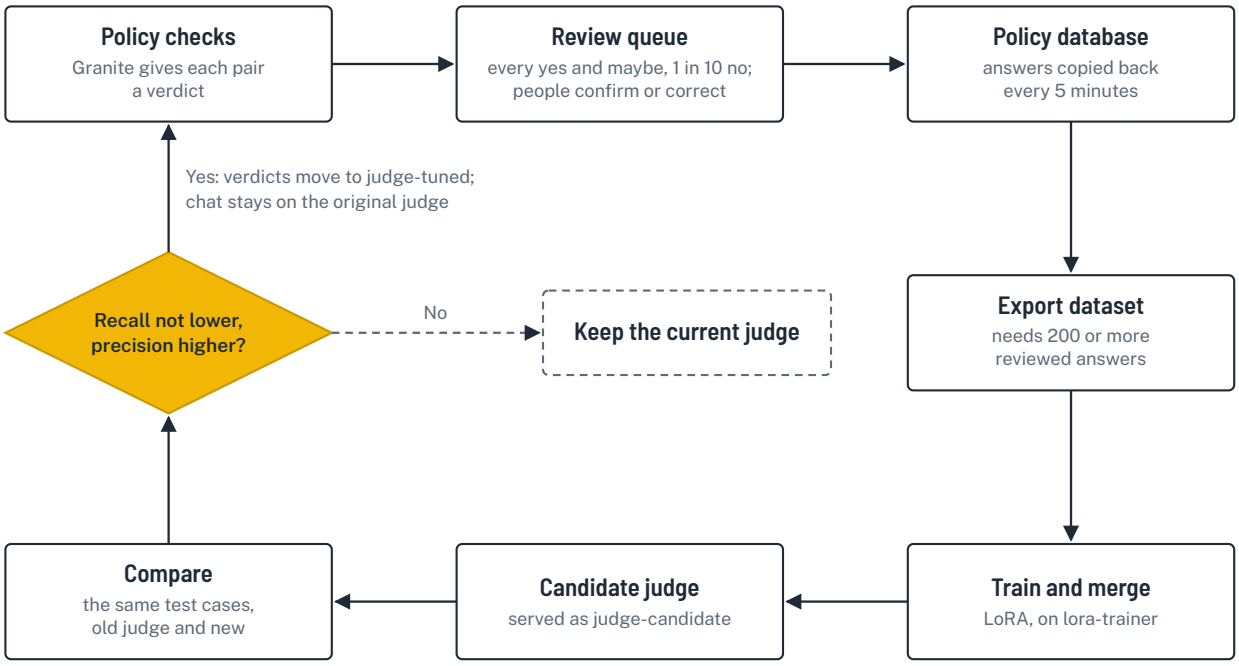
Chapter 6

Review queue and judge training

Let people check answers, then teach the judge from them.

For: Administrators, reviewers and engineers

People check the judge's answers in a review queue (Argilla). Their corrections are saved and, once there are enough, used to fine-tune the judge. Nothing leaves your servers: everything runs on the sealed `backend` network, with telemetry and internet access turned off.



The review and training loop. A new judge replaces the old one only if it scores better.

For the administrator

What you get

- **Review queue**: one card per "this section may be affected" answer. The reviewer sees the old and new wording, the other policy's section and the program's answer, and presses Submit.
- **Training**: after a few hundred reviewed cards, an engineer can teach the judge from them. A new judge only replaces the old one if it scores better on your test cases.

Two kinds of review

This version has two separate ways a person can settle a "maybe". They do not feed each other.

	Chat decision (`REVIEW_GATE=1`)	Review queue (Argilla)
When	While the check is paused, before the final answer	Any time after the check, in a separate screen
How	The person types `decide <check number> ...` in the chat; the upload filter records it	A reviewer opens a card and presses Submit
Changes	The final answer and Excel file for that check ("maybe" becomes "yes" or is dropped)	The `judgements` rows: this is what training reads
Training data	**No.** The decision is kept on the check only	**Yes**

The assistant has no tool to record either kind. See ui.md for the chat decision.

Start the review queue

1. In `.env`, fill in the "Review queue (Argilla)" block. Make each secret with `openssl rand -hex 16` (use letters and digits only).
2. Start it: `docker compose --profile review up -d`. The first start takes a few minutes.
3. Ask whoever runs the web proxy to add the Argilla address (see "Engineer: proxy" below).
4. Open `http://<your-server>:6900` and sign in as the admin user from `.env`.

Add a reviewer

Run once per person (change the name, username and password):

```
docker compose exec argilla python -m argilla_server database users create \
  --first-name "Ana" --username ana --password "a-long-password" --role annotator --workspace
policy-review
```

Give them the address, their username and password. A reviewer only sees their own cards. To add a reviewer who can also see everyone's work, use `--role admin` instead of `annotator`.

What reviewers do

Reviewers need no training. The first screen shows the instructions in plain words. For each card:

1. Read the old and new wording, then the section that may be affected.
2. The program's answer is already filled in. If it is right, press **Submit**.
3. If it is wrong, change the answer, pick the kind of impact, add a short note saying why, and press **Submit**.
4. **Can't tell** is allowed. **Discard** skips a card. Nothing is ever lost or published.

Ask reviewers to correct the program when it is wrong, not to agree by default. Corrections are the most useful part.

How answers come back

The sync job in policy-api sends new answers to the queue and copies submitted answers back into the policy database (`reviewer_decision`, `reviewer_impact_type`, `reviewer_note`, `reviewed_by`, `reviewed_at`). A card that is only a draft or was discarded is ignored.

Turn the sync on by setting `ARGILLA_URL=http://argilla:6900` in `.env` (with `ARGILLA_API_KEY` already set) and restarting policy-api. It runs every 5 minutes (`REVIEW_SYNC_SECONDS`) as a job in the job queue. It sends every "yes" and "maybe" from finished checks, plus 1 in 10 of the "no" answers (`REVIEW_SAMPLE_NO`), because reviewers also need to catch what the program missed. Each card shows the clause exactly as the program saw it, even if the policy has changed since.

When is there enough?

Training refuses to start below **200 reviewed answers** and tells you how many more it needs. More is better: aim for 500 or more, covering at least 5 different changed policies, and include plenty of corrections, not only agreements. Check the count any time:

```
docker compose --profile train run --rm lora-trainer export_dataset.py --out /out/data
```

It prints the counts and writes nothing until there is enough.

Back up and delete

The queue's data lives in the `argilla-pgdata`, `argilla-esdata` and `argilla-data` volumes. Back up `argilla-pgdata` with the policy database. The index (`argilla-esdata`) can be rebuilt. Reviewer answers are also copied into the policy database, which is the record that counts.

Engineer

Services (all on `backend`, none publish a port)

Service	What	Profile
`argilla`, `argilla-worker`	Argilla v2.8.0 server and background worker (UI port 6900 inside the network)	review
`argilla-db`	Postgres 16.6, Argilla's own data, kept apart from the policy database	review
`argilla-es`	Elasticsearch 8.17.0, search index	review
`argilla-redis`	Redis 7.4.2, job queue	review
`lora-trainer`	exports data, trains and merges (image built from `training/Dockerfile`)	train
`judge-candidate`	vLLM serving the merged model as `judge-candidate`	train

Versions are set in `.env` (`ARGILLA_TAG`, `ARGILLA_ES_TAG`, `ARGILLA_REDIS_TAG`, `ARGILLA_PG_TAG`). The Python client `argilla==2.8.0` in `policy_api/requirements.txt` must match `ARGILLA_TAG`. Telemetry is off: `HF_HUB_DISABLE_TELEMETRY=1`, `HF_HUB_OFFLINE=1`, `ARGILLA_ENABLE_TELEMETRY=0`.

`argilla` refuses to start if `ARGILLA_ADMIN_PASSWORD` or `ARGILLA_API_KEY` is empty, and the password must be 8+ characters. The first start creates the owner account and workspace from `.env`. Restarting is safe (the account is not created twice). Its healthcheck uses `/api/docs`: in v2.8.0 `/api/v1/status` returns 422 even when healthy.

Proxy: expose the Argilla UI

Reviewers reach Argilla through the existing proxy. Add this `server` block to `proxy/nginx.conf` (next to the Open WebUI one), and publish the port on the `proxy` service: `- "${BIND_ADDR:-127.0.0.1}:6900:6900"`.

```
  # Argilla (review queue)
  server {
    listen 6900;
    client_max_body_size 25m;
    location / {
      set $argilla http://argilla:6900;
      proxy_pass $argilla;
      proxy_http_version 1.1;
      proxy_set_header Host $host;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
      proxy_set_header X-Forwarded-Proto $scheme;
    }
  }
```

policy-api talks to Argilla directly on the backend network, never through the proxy: `ArgillaReview(api_url="http://argilla:6900", api_key=ARGILLA_API_KEY, workspace=ARGILLA_WORKSPACE)` (`policy_api/app/review.py`). Calls are synchronous; call them with `asyncio.to_thread`.

What goes in and out

- `push(items)`: one record per judgement, record id = `str(judgements.id)`. Cards already in the queue are skipped, so a reviewer's work is never overwritten. The program's answer is pre-filled.
- `pull_submitted(since=None)`: only responses with status `submitted`. If two people answered one card, the latest wins. `no` always comes back with impact type `none`. Pass the newest `reviewed_at` you have stored as `since` to pull only new answers.
- Training reads the same rows from Postgres: `reviewer_decision` plus `prompt`, so `prompt` must be the exact text the judge saw (`render_pair`).

Train

All commands run from the repo root. Granite runs on its own server, so the GPUs here only need to hold the training job: give `lora-trainer` (and later `judge-candidate`) GPUs in `compose.gpu.yaml`, like the others:

```
  lora-trainer:
    deploy: {resources: {reservations: {devices: [{driver: nvidia, device_ids: ["0"],
capabilities: [gpu]}]}}}
  judge-candidate:
    deploy: {resources: {reservations: {devices: [{driver: nvidia, device_ids: ["1","2"],
capabilities: [gpu]}]}}}
```

```
docker compose --profile train build lora-trainer          # once; needs internet, or build
elsewhere and `docker load`
docker compose --profile train run --rm lora-trainer export_dataset.py --out /out/data
docker compose --profile train run --rm lora-trainer train_lora.py \
    --train /out/data/train.jsonl --eval /out/data/holdout.jsonl --base /models/gpt-oss-20b --out
/out/adapter
docker compose --profile train run --rm lora-trainer merge_adapter.py \
    --base /models/gpt-oss-20b --adapter /out/adapter --out /out/merged
```

Outputs are in `training/out/` (not in git: they contain policy text). `training/README.md` has the settings and the details.

Evaluate and promote

The rule: **switch only if recall is not lower and precision is higher** on the same test cases.

1. Start the candidate: `docker compose --profile train up -d judge-candidate` (waits for the model to load).
2. Run the same cases against both judges:

```
docker compose exec policy-api python -m app.evaluate /eval/cases.json > training/out/base.json
docker compose exec -e JUDGE_URL=http://judge-candidate:8000/v1 -e JUDGE_MODEL=judge-candidate \
    policy-api python -m app.evaluate /eval/cases.json > training/out/candidate.json
python3 training/compare_eval.py training/out/base.json training/out/candidate.json
```

`app.evaluate` already reads `JUDGE_URL` and `JUDGE_MODEL` from the environment, so it needs no change. Keep `eval/cases.json` free of the policies used for training (it is a separate answer key). 3. If `compare_eval.py` prints `RESULT: the new model is better` (exit code 0), switch.

Switch. The tuned model gets its own service, `judge-tuned`. Only the policy checks' verdicts move to it; the chat in Open WebUI stays on Granite, so a model trained only to give verdicts never has to hold a conversation.

```
docker compose --profile train stop judge-candidate
mv training/out/merged models/judge-tuned-2026-11        # any name; keep one folder per version
```

In `.env` (and assign it GPUs with `python3 scripts/setup_gpus.py`, answering for `judge-tuned`):

```
TUNED_JUDGE_DIR=judge-tuned-2026-11
JUDGE_URL=http://judge-tuned:8000/v1
JUDGE_MODEL=judge-tuned
```

```
docker compose --profile tuned up -d judge-tuned      # wait until `docker compose ps judge-
tuned` is healthy
docker compose up -d policy-api
```

Run one real check from Open WebUI. Every verdict records the model that gave it, so the `judgements` table shows which checks used the tuned judge.

Roll back (seconds, no model reload): delete or comment out the `JUDGE_URL` and `JUDGE_MODEL` lines in `.env`, then `docker compose up -d policy-api`. Stop `judge-tuned` when you no longer need it.

The Granite server is never replaced; the tuned model only takes over the verdicts.

Known limits

- The judge is now Granite 4.2 30B, but these scripts still fine-tune gpt-oss-20b. A tuned gpt-oss-20b can still be tried as `judge-candidate` and must beat Granite on the test cases to replace it. Tuning Granite itself needs `train_lora.py` changed for a dense bf16 model (no `Mxfp4Config`, LoRA on attention and MLP layers) and is not done yet.
- vLLM cannot apply LoRA to gpt-oss mixture-of-experts layers, so training is attention-only by default and the adapter is always merged into a normal model folder before serving.
- The merged model is bf16: about 42 GB for 20b and about 240 GB for 120b, so `judge-candidate` needs correspondingly more GPU memory than the shipped 4-bit model. Re-quantizing is optional and not done here.
- Training on reviewer answers teaches the judge their habits, including mistakes. The test cases are the check; never skip the promotion rule.

Chapter 7

Evaluation

Measure whether it finds the right policies.

For: Policy owners and engineers

Testing shows the code works. **Evaluation shows whether it finds the right policies.** Do it before real use, and again after every change of model, prompt, embedding model or search settings.

What to measure

Measure	Meaning	Target to start with
Recall	Of the policies that really needed updating, how many did it flag (Yes or Maybe)?	**≥ 0.95**. Missing one is the costly error
Precision	Of the policies it flagged, how many really needed updating?	≥ 0.5 at first. Each false flag costs a reviewer a few minutes
Candidate recall	Did the truly affected clauses even reach the judge (step 3)?	≥ 0.98. If a clause never reaches the judge, no model can flag it
Maybe rate	Share of flagged policies marked Maybe rather than Yes	Watch it fall as the model improves
Unreadable verdicts	Verdicts with "could not be read"	0

Set your own targets with the policy owners. Write them down before you look at results.

Build the case set

Each case is a **real past change** to a head policy, with the list of policies that people actually had to update afterwards.

```
{
  "name": "POL-007 2025 revision",
  "head_policy": "POL-007",
  "subset": "Safety",
  "new_version_file": "private/POL-007-v4.docx",
  "expected_impacted": ["SAF-112", "SAF-130", "SAF-150"],
  "source": "Change request CR-2025-114; updated policies listed in the closure note"
}
```

Where the answers come from matters more than how many cases you have:

- **Use the record of what was actually updated** (change requests, document control logs, revision histories), not a fresh list someone writes while looking at this system's output.
- **Load the policy library as it was before the change.** Otherwise the affected policies already contain the new wording, and the run finds nothing to flag.

- **Keep a separate hold-out set.** Tune `SEMANTIC_K`, `RERANK_KEEP` and prompts on one set of cases, and report results on a different set you never looked at while tuning. A score on the cases you tuned against tells you little.
- **Include hard cases on purpose:** a renamed defined term, a renumbered section, a removed section, a policy that's affected but never cites the head policy, and a change that affects nothing.
- Start with **10–20 cases**, and grow to 50+ over time. Store them in `eval/private/` and `eval/cases.json`, which git ignores.

Historical records are imperfect: people sometimes missed policies too. When the system flags a policy that the record doesn't list, have the policy owner decide. If it really was affected, fix the case and note it in `source`. This is often the most useful finding.

Run it

```
docker compose exec policy-api python -m app.evaluate /eval/cases.json --min-recall 0.95
```

- By default, Yes **and** Maybe count as found, because both reach a person. `--yes-only` measures how often the model is confident and right.
- The command exits non-zero if recall is below `--min-recall` or any case errored, so it can gate a release.
- The output lists, for every case, which expected policies were **missed** and which **extra** ones were flagged.

Run evaluation against a **test database** loaded with the historical policy versions, never the live one.

Finding why something was missed

For each missed policy, check where it was lost:

```
-- Did any of its clauses reach the judge?
SELECT candidate_number, found_by, stage, result->>'impacted', result->>'why'
FROM judgements WHERE run_id = '<run>' AND candidate_policy = '<missed id>';
```

What you see	Cause	Fix
No rows	It never became a candidate	Check that it's in the group; add explicit references (`--llm-refs`); raise `SEMANTIC_K` / `RERANK_KEEP`; try another embedding model
Rows, all "no"	The judge decided wrongly	Read the pair. Adjust the judge prompt in `app/prompts.py`; turn on escalation; later, fine-tune
Rows, "unsure" turned "no" by escalation	The larger model overruled	Compare both verdicts; decide which model to trust for this kind of change
The changed section wasn't detected	Parsing or diff problem	Check the file's numbering (how-to.md)

Comparing models or settings

Change **one thing at a time** and run the same case set. Record the results in a table like this:

Date	Judge	Escalation	Embed / rerank	K / keep	Recall	Precision	Maybe rate	Time per run

Small case sets are noisy. A difference of one case in 15 is about 7 points of recall. Don't adopt a change on a gain smaller than that unless it holds on the hold-out set.

After launch

- Every reviewer decision on a Maybe is a new labelled example. They are collected through the review queue; see review-and-training.md.
- Each quarter, rerun the evaluation and add new real changes to the case set.
- Before fine-tuning, keep the hold-out cases out of the training data.

Chapter 8

Testing

Run and understand the tests.

For: Engineers

Tests answer one question: **does the code do what it claims?** Whether the *models* find the right policies is a separate question, answered by evaluation.md.

Levels

Level	Command	Needs	Proves
1. Unit + flow	`pytest -q` (in `policy_api/`)	Python only	Parsing, diff, candidate search, verdict handling, chat answer, Excel, API auth and flow
2. Postgres	`TEST_DATABASE_URL=… pytest -q`	Docker	The real store: SQL, pgvector search, versions, reference graph, audit rows, restart handling
3. Stack (demo)	`DEMO_MODE=1` + Compose	Docker	Images build, services start, proxy rules, download links, network containment
4. Stack (real models)	Smoke test below	GPU server	vLLM and TEI wiring, structured output, tool calling through Hermes
5. Evaluation	`python -m app.evaluate`	GPU server + labelled cases	Whether it finds the right policies (evaluation.md)

Run levels 1–2 on every change. Run 3 before every release. Run 4–5 after any model, image or prompt change.

Level 1: unit and flow tests

```
cd policy_api
python3 -m venv .venv && .venv/bin/pip install -r requirements-dev.txt
.venv/bin/python -m pytest -q
```

File	Covers
`test_documents.py`	Clause splitting (headings, list items, duplicate numbers), reference and defined-term extraction, front matter, diff (modified, added, removed, renumbered)
`test_pipeline.py`	The full sample run with stand-in models: which policies are Yes/Maybe/not affected, evidence, counts, the chat answer, workbook sheets, **policy text can't become an Excel formula**
`test_api.py`	API key required, **wrong Host header refused** (DNS-rebinding guard), lookups, upload → run → download, tampered download link refused, helpful errors

The stand-in models in `app/fakes.py` are deterministic. The fake judge says "yes" when a linked clause still contains wording the head policy removed, "unsure" when a clause is linked but has no such wording, and "no" otherwise. That makes flow tests exact. It does **not** make them evidence that the real models work.

Level 2: Postgres

```
docker run -d --name sk-test-db -e POSTGRES_PASSWORD=test -p 127.0.0.1:55432:5432 \
  -v "$PWD/../db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro" pgvector/pgvector:pg17
TEST_DATABASE_URL=postgresql://postgres:test@localhost:55432/postgres .venv/bin/python -m pytest
-q
docker rm -f sk-test-db
```

`test_pgstore.py` truncates its tables, so never point `TEST_DATABASE_URL` at a real database.

Level 3: the stack in demo mode

Uses the sample policies and stand-in models, so no GPUs are needed. Make a throwaway env file (secrets from `openssl rand -hex 32`, any image tags, and `DEMO_MODE=1`), then:

```
docker compose --env-file /tmp/test.env up -d --build db policy-api proxy
curl -s http://127.0.0.1:8100/health                        # {"ok":true,"demo_mode":true}
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8100/subsets   # 404: API routes not
exposed
COMPOSE_ARGS="--env-file /tmp/test.env" scripts/check_containment.sh     # PASS contained
docker compose --env-file /tmp/test.env exec policy-api python -m app.evaluate
/eval/cases.example.json --demo
docker compose --env-file /tmp/test.env down -v
```

Testing the Open WebUI tool without Open WebUI (other versions only)

This version does not use the Open WebUI tool; Hermes and its `secretary` plugin do that job. The file stays in the repo for the `main` and `langgraph` versions. Its logic can be run inside the policy-api container, which has the same libraries:

```
docker cp openwebui/policy_impact_tool.py secretary-keeper-policy-api-1:/tmp/tool.py
docker compose exec -T -e KEY=<POLICY_API_KEY> policy-api python3 - <<'EOF'
import asyncio, os, sys; sys.path.insert(0, "/tmp")
from tool import Tools
t = Tools(); t.valves.API_KEY = os.environ["KEY"]
files = [{"name": "POL-007-v4.md", "file": {"path": "/samples/changes/POL-007-v4.md"}}]
print(asyncio.run(t.check_policy_change("Safety", __files__=files)))
EOF
```

Also try: an unknown group, an ambiguous policy name, no file and no policy, and a wrong `API_URL`. Each must return a plain instruction for the chat model, never a stack trace.

Level 4: smoke test with the real models

On the GPU server, after setup.md:

1. From inside the network, `docker compose exec policy-api python3 -c "import httpx; print(httpx.get('http://granite-gateway:8000/v1/models').json())"` lists `granite-4.2-30b` (the gateway adds the key).
2. Ingest `samples/policies`, upload `samples/changes/POL-007-v4.md` in Open WebUI, and ask about Safety.
3. Expect SAF-112 and SAF-130 flagged, and SAF-141 and SAF-160 not affected. SAF-150 may be Yes or Maybe.
4. In `judgements`, every `result` has all six fields, and none has the "could not be read" reason.
5. Ask something ambiguous ("check the lockout thing") and confirm the assistant asks one question back.

Then ingest a copy of your real policies **in a test database** and run the evaluation.

Testing the tests

A test that can't fail proves nothing. When you add or change a check, break the thing it guards once and confirm the check fails:

- The containment script must fail when a service *can* reach the internet. During development it did: Docker Desktop ignored the "no outbound NAT" network setting, which is why policy-api and Open WebUI now sit only on the sealed `backend` network.
- It must also fail when it checked nothing. An early version printed "PASS" because it couldn't read the env file and skipped every service. It now fails if zero services were checked, and each check starts with a positive control (reaching an internal service).
- The formula-injection test checks the cell's *type*, not just its text.